

1. Sentiment Analysis - Problem

Sentiment Analysis, also known as **opinion mining**, is a sub-field of **Natural Language Processing (NLP)** that aims to determine the **attitude** (or sentiment) of a speaker, writer, or other subject with respect to some topic or the overall contextual polarity of a document.

The Core Problem

The main problem sentiment analysis addresses is: **Given a piece of text (e.g., a review, a tweet, a news article), how do we automatically classify the underlying opinion expressed within it?**

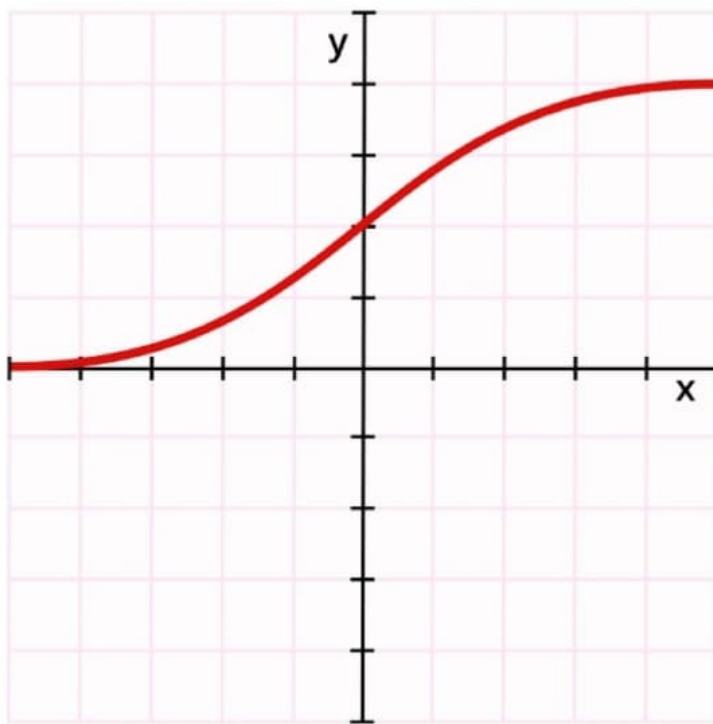
- **Classification:** It is fundamentally a **text classification** problem. The goal is to map the text input x to an output category y (the sentiment label).
- **Typical Categories:**
 - **Binary:** Positive / Negative.
 - **Ternary (Most Common):** Positive / Negative / Neutral.
 - **Granular:** Ranging from "Very Positive" to "Very Negative" (often using a 5-star scale).
- **Key Challenges:**
 - **Subtlety & Sarcasm:** Detecting non-literal language (e.g., "Great, my flight was cancelled...").
 - **Context Dependence:** A word like "long" is good in a "long battery life" review but bad in a "long wait time" review.
 - **Domain Specificity:** Models trained on movie reviews may perform poorly on financial news sentiment.
 - **Neutrality:** Distinguishing between truly neutral text and text that is simply difficult to classify.

2. Logistic Regression Intuition 🧠

Logistic Regression is a linear model used for **binary classification**. Despite the name "regression," it is a classification algorithm. Its core intuition lies in using a special function to squash the output of a linear equation into a probability.

Getty Images

Logistic Function



The Intuition

1. **Linear Combination:** The process starts by calculating a linear score (or log-odds) z based on the input features $\mathbf{x}$ and learned weights $\mathbf{w}$ (and bias b):

$$z = \mathbf{w}^T \mathbf{x} + b$$

This score z can range from $-\infty$ to $+\infty$.

2. **The Sigmoid/Logistic Function:** Since we need a probability $P(y = 1|\mathbf{x})$, which must be between 0 and 1, we pass the linear score z through the **Sigmoid (or Logistic) function** σ :

$$P(y = 1|\mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

3. Probability Interpretation:

- If z is a large **positive** number, $\sigma(z)$ approaches 1 (high probability of positive class).
- If z is close to **zero**, $\sigma(z)$ approaches 0.5 (uncertainty).
- If z is a large **negative** number, $\sigma(z)$ approaches 0 (low probability of positive class, high probability of negative class).

4. **Classification:** A **decision boundary** is set, usually at $P = 0.5$. If $P(y = 1|\mathbf{x}) \geq 0.5$, the input is classified as class 1 (e.g., Positive); otherwise, it's classified as class 0 (e.g., Negative). The line or plane where $P = 0.5$ is the decision boundary defined by $\mathbf{w}^T \mathbf{x} + b = 0$.

3. Multiclass Logistic Regression

While standard Logistic Regression is inherently binary, it can be extended to handle classification problems with **more than two classes** (e.g., Positive, Neutral, Negative). This extension is typically achieved using two main strategies: **One-vs-Rest (OvR)** or the more mathematically elegant **Softmax Regression**.

A. One-vs-Rest (OvR)

- **Mechanism:** For K classes, you train K independent binary Logistic Regression models.
- **Model j :** Model j is trained to distinguish between **Class j** (the positive class) and all other classes combined (the negative class).
- **Prediction:** For a new input $\mathbf{x}$, each of the K models outputs a probability. The final prediction is the class j that yields the **highest predicted probability**.

$$\text{Prediction}(\mathbf{x}) = \underset{j}{\operatorname{argmax}} (P_j(\text{Class } j|\mathbf{x}))$$

B. Softmax Regression (Multinomial Logistic Regression)

- **Mechanism:** This is a direct generalization. Instead of using the Sigmoid function, it uses the **Softmax function** to ensure the output probabilities for all classes sum up to 1.
- **Linear Scores:** For K classes, a weight vector $\mathbf{w}_j$ is learned for each class j . The linear score for class j is $z_j = \mathbf{w}_j^T \mathbf{x} + b_j$.
- **Softmax Function:** The probability of an instance belonging to class j is:

$$P(y = j|\mathbf{x}) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

- **Advantage:** The outputs are a proper probability distribution, making it the preferred method for true multiclass problems.

4. Logistic Regression Training and Interpretation

Training Logistic Regression involves finding the optimal weight vector $\mathbf{w}$ that minimizes the error between the predicted probabilities and the actual labels. Interpretation is key to understanding the model's decisions.

A. Training: Finding $\mathbf{w}$

1. **Cost Function (Loss):** Logistic Regression uses the **Cross-Entropy Loss** (or Log-Loss) function because the squared error loss is non-convex for this model, making it difficult to optimize. For a single sample $(\mathbf{x}, y)$:

$$J(\mathbf{w}) = -[y \log(P(y = 1|\mathbf{x})) + (1 - y) \log(1 - P(y = 1|\mathbf{x}))]$$

This penalizes the model heavily when it is highly confident in the wrong prediction.

2. **Optimization:** The optimal $\mathbf{w}$ is found by minimizing the average cost over all training examples using an iterative algorithm, typically **Gradient Descent**.
 - **Gradient Descent:** Iteratively adjusts the weights in the direction opposite to the gradient of the cost function (the steepest descent).

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \eta \nabla J(\mathbf{w})$$

where η is the learning rate and $\nabla J(\mathbf{w})$ is the gradient.

3. **Regularization (Optional but common):** L1 or L2 penalties are often added to the cost function to prevent overfitting by keeping the weights small.



B. Interpretation

The weights w are highly interpretable in Logistic Regression because the model is linear.

- **Weight Magnitude:** The magnitude of a weight w_i associated with feature x_i indicates the importance of that feature in the classification decision.
- **Weight Sign (Direction):**
 - A **positive** weight w_i means that an increase in the feature value x_i increases the linear score z , which in turn **increases the probability** $P(y = 1|\mathbf{x})$. For sentiment analysis, this feature is associated with **Positive** sentiment.
 - A **negative** weight w_i means that an increase in x_i **decreases the probability** $P(y = 1|\mathbf{x})$. This feature is associated with **Negative** sentiment.

Example for Sentiment: If the model is classifying (1=Positive, 0=Negative), a high positive weight for the feature "excellent" and a high negative weight for the feature "disappointed" are expected.